

API REST de Productos

Documentación técnica y plan de pruebas — Productos con imagen polimórfica uno a uno (Laravel 12)

Proyecto	store-api — API REST de Productos
Alumno	Eduardo Hernández Gutiérrez
Grupo	TIID_01_05
Docente	Miguel Ángel Montoya Cerro
Fecha	10 de julio de 2026
Puntaje total	100 puntos

Contenido

1. Objetivo
2. Arquitectura y ambiente
3. Estructura de datos
4. Modelos y relación polimórfica
5. Endpoints de la API
6. Validaciones
7. Evidencia visual: galería de productos
8. Plan de pruebas (Plantilla 12.1)
9. Casos de prueba (Plantilla 12.2)
10. Matriz de seguimiento (Plantilla 12.3)
11. Reporte de defectos (Plantilla 12.4)
12. Tipos de prueba propuestos
13. Conclusión técnica

1. Objetivo

Documentar el desarrollo y la verificación de una API REST en Laravel para administrar **productos**, cada uno con una **imagen principal** asociada mediante una relación **polimórfica uno a uno** (`morphOne` / `morphTo`). Se comprueba el funcionamiento de los cinco métodos principales (index, show, store, update, destroy), sus validaciones y el manejo de la imagen, y se documenta un plan de pruebas con evidencias reales de ejecución.

Laravel 12.63.0

PHP 8.2.12

PostgreSQL 18

Relación polimórfica 1:1

10 casos de prueba · 100% aprobados

2. Arquitectura y ambiente

El proyecto sigue la arquitectura estándar de Laravel: rutas → *FormRequest* (validación) → controlador → modelo Eloquent → base de datos, respondiendo siempre en formato JSON. La imagen se guarda en el disco público y se sirve mediante un enlace simbólico.

Componente	Detalle
Framework	Laravel Framework 12.63.0
Lenguaje	PHP 8.2.12 (ZTS, x64)
Base de datos	PostgreSQL 18.3 — base <code>ecommerce_db</code>
Sistema operativo	Windows 11 (build 10.0.26200)
Servidor local	<code>php artisan serve</code> — <code>http://127.0.0.1:8000</code>
Almacenamiento de imágenes	Disco <code>public</code> (<code>storage/app/public/products</code>) enlazado con <code>php artisan storage:link</code>
Herramienta de prueba	cURL (peticiones HTTP reales contra el servidor)
Formato de respuesta	JSON (<code>application/json</code>)

Estructura del proyecto (archivos clave)

```
store-api/
├── app/
│   ├── Http/
│   │   ├── Controllers/Api/ProductController.php    # index, show, store, update, destroy
│   │   ├── Requests/StoreProductRequest.php         # validación al crear
│   │   └── UpdateProductRequest.php                 # validación al actualizar
│   └── Models/Product.php · Image.php               # morphOne / morphTo
├── database/migrations/ ... _create_products_table.php
│   └── ... _create_images_table.php                 # $table->morphs('imageable')
└── routes/api.php                                  # rutas /api/products
```

3. Estructura de datos

Se definieron dos tablas. La tabla `images` incluye las columnas polimórficas `imageable_id` e `imageable_type` generadas por el helper `$table->morphs('imageable')`, lo que permite que en el futuro el mismo modelo `Image` se reutilice con otros módulos.

Tabla	Campos	Descripción
products	<code>id</code> , <code>name</code> (string), <code>description</code> (text), <code>price</code> (decimal 10,2), <code>stock</code> (integer), <code>created_at</code> , <code>updated_at</code>	Información principal del producto.
images	<code>id</code> , <code>path</code> (string), <code>imageable_id</code> (bigint), <code>imageable_type</code> (string), <code>created_at</code> , <code>updated_at</code>	Imagen principal asociada al producto mediante relación polimórfica.

```
// database/migrations/ ... _create_images_table.php
Schema::create('images', function (Blueprint $table) {
    $table->id();
    $table->string('path');
    $table->morphs('imageable');    // crea imageable_id + imageable_type
    $table->timestamps();
});
```

4. Modelos y relación polimórfica

El producto declara la relación `morphOne` (una sola imagen principal) y la imagen declara `morphTo` . El modelo `Image` añade un accesor `url` que expone la ruta pública del archivo.

```
// app/Models/Product.php
class Product extends Model {
    protected $fillable = ['name', 'description', 'price', 'stock'];

    public function image() {
        return $this->morphOne(Image::class, 'imageable');    // 1 : 1
    }
}
```

```
// app/Models/Image.php
class Image extends Model {
    protected $fillable = ['path'];
    protected $appends = ['url'];

    public function imageable() {
        return $this->morphTo();
    }
    public function getUrlAttribute() {
        return asset(Storage::url($this->path));
    }
}
```

¿Por qué polimórfica? Con `imageable_type` almacenando el nombre de la clase (`App\Models\Product`), el mismo modelo `Image` podría asociarse mañana a otros modelos (por ejemplo `User` o `Category`) sin cambiar la tabla ni crear nuevas llaves foráneas.

5. Endpoints de la API

Las rutas se registraron en `routes/api.php` , por lo que quedan bajo el prefijo `/api` .

Método	Ruta	Acción	Resultado
GET	<code>/api/products</code>	index	Lista todos los productos con su imagen.
GET	<code>/api/products/{id}</code>	show	Detalle de un producto con su imagen.
POST	<code>/api/products</code>	store	Crea un producto con datos e imagen.
POST	<code>/api/products/{id}</code>	update	Actualiza datos y, si aplica, la imagen.
DELETE	<code>/api/products/{id}</code>	destroy	Elimina el producto y su imagen (registro + archivo).

Nota de diseño sobre `update` . La actualización se implementó con `POST /api/products/{id}` en lugar de `PUT/PATCH` . PHP no interpreta el cuerpo `multipart/form-data` (necesario para subir archivos) en peticiones `PUT/PATCH`; por ello se usa `POST`, patrón habitual en Laravel para formularios con archivos. En la sección 11 (defecto D-02) se documenta esta desviación respecto a la ruta sugerida y su mitigación.

El manejo de la imagen en `store()` guarda el archivo y crea el registro polimórfico dentro de una transacción, revirtiendo el archivo si algo falla:

```
// ProductController@store (extracto)
$product = Product::create($request->only(['name', 'description', 'price', 'stock']));
$path = $request->file('image')->store('products', 'public');
$product->image()->create(['path' => $path]); // relación polimórfica
```

6. Validaciones

Se usan *Form Requests*: `StoreProductRequest` (todos los campos requeridos) y `UpdateProductRequest` (campos opcionales con `sometimes`).

Campo	Regla	store	update
name	<code>string max:255</code>	required	sometimes
description	<code>string</code>	required	sometimes
price	<code>numeric min:0</code>	required	sometimes
stock	<code>integer min:0</code>	required	sometimes
image	<code>image mimes:jpg,jpeg,png max:2048</code>	required	sometimes

7. Evidencia visual: galería de productos

La siguiente captura muestra la respuesta real de `GET /api/products` renderizada como tarjetas: cada producto aparece con su **imagen principal real** servida desde `storage`, lo que demuestra que la relación polimórfica funciona de extremo a extremo.

Galeria de productos — relacion polimorfica Product → Image

GET <http://127.0.0.1:8000/api/products> · datos y 9 imágenes reales servidas desde storage/app/public

HTTP 200 · 9 productos



#2

iPhone 16

\$31999.00

stock: 12



#3

Alexa

\$1000.00

stock: 35



#4

Galaxy Buds 3

\$4999.00

stock: 28



#5

Logitech G

\$2499.00

stock: 23



#6

Ipad Air

\$2099.00

stock: 10



#7

Lenovo TinkPad

\$2299.00

stock: 8



#8

XREAL Air 2

\$1899.00

stock: 13

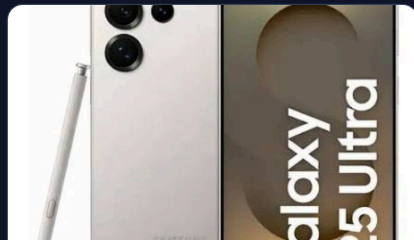


#9

Apple Watch 10

\$9999.00

stock: 20



#10

Samsung Galaxy S25

\$2499.00

stock: 17

Cada tarjeta demuestra que el producto (products) tiene asociada una imagen (images) via morphOne / morphTo · imageable_type = App\Models\Product

GET /api/products — 9 productos con su imagen real (HTTP 200).

8. Plan de pruebas (Plantilla 12.1)

Nombre del proyecto	API REST de Productos (store-api).
Objetivo de las pruebas	Comprobar la creación, consulta, actualización y eliminación de productos con imagen, así como sus validaciones y el manejo del archivo.
Alcance	Endpoints index, show, store, update y destroy, y la relación Product–Image (morphOne / morphTo).
Fuera de alcance	Autenticación, frontend, pagos, inventario avanzado y notificaciones (no solicitados en esta etapa).
Ambiente de prueba	Windows 11; PHP 8.2.12; Laravel 12.63.0; PostgreSQL 18.3 (ecommerce_db); servidor <code>php artisan serve</code> ; herramienta cURL.
Datos de prueba	Productos válidos (name, description, price, stock), imagen JPG válida (<code>valid_product.jpg</code>), archivo no imagen (<code>no_es_imagen.txt</code>), valores inválidos (price=-50, stock=-3) e id inexistente (999999).
Tipos de prueba	API, funcional, validación, integración, sistema y aceptación (manual con cURL).
Criterios de aceptación	Códigos HTTP correctos (200/201/404/422), JSON esperado, datos guardados en BD y relación con imagen funcionando.
Criterios de rechazo	Errores 500, validaciones ausentes, datos incorrectos, imagen no asociada o endpoint sin respuesta correcta.
Responsables	Alumno (desarrollo, ejecución de pruebas y evidencias) / QA.
Evidencias	Capturas de petición/respuesta JSON reales, galería de imágenes y respuestas de error. Ver secciones 7 y 9.

9. Casos de prueba (Plantilla 12.2)

Se documentan los **8 casos mínimos obligatorios** (CP-01 a CP-08) más **2 casos adicionales** (CP-09 actualización de imagen y CP-10 valores inválidos). Cada caso incluye la captura real de la petición y la respuesta de la API.

ID	CP-01
Nombre	Crear producto con imagen válida
Prioridad	Alta
Tipo de prueba	API / Funcional
Endpoint	<code>POST /api/products</code>
Objetivo	Verificar que la API crea un producto con sus datos y una imagen principal, guardando el archivo y la relación polimórfica.
Precondiciones	Servidor activo y base de datos migrada.
Datos de entrada	form-data: name, description, price=24999.90, stock=8, image=valid_product.jpg (JPG válido).
Pasos	1) Seleccionar método POST y URL /api/products. 2) Enviar los campos como form-data incluyendo el archivo image. 3) Ejecutar y observar la respuesta.
Resultado esperado	HTTP 201 Created, JSON con message='Producto creado correctamente.' y el producto creado con su objeto image (id, path, url, imageable_type, imageable_id).
Resultado real	HTTP 201. Se creó el producto id=11 y su imagen id=11 con imageable_type=App\Models\Product. El archivo se almacenó en storage/app/public/products.
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-01

Crear producto con imagen valida

POST http://127.0.0.1:8000/api/products

● Status: **201 Created**

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: **Accept: application/json**

name	Laptop Gamer Legion 5	Text
description	Laptop para videojuegos con RTX 4060 y 16GB RAM	Text
price	24999.90	Text
stock	8	Text
image	 valid_product.jpg	File

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "message": "Producto creado correctamente.",
  "data": {
    "name": "Laptop Gamer Legion 5",
    "description": "Laptop para videojuegos con RTX 4060 y 16GB RAM",
    "price": "24999.90",
    "stock": "8",
    "updated_at": "2026-07-10T15:09:20.000000Z",
    "created_at": "2026-07-10T15:09:20.000000Z",
    "id": 11,
    "image": {
      "id": 11,
      "path": "products/gCCwI8vdo9b333MwFFyYZ4ISpYXbq9CGpFLrEdEO.jpg",
      "imageable_type": "App\\Models\\Product",
      "imageable_id": 11,
      "created_at": "2026-07-10T15:09:20.000000Z",
      "updated_at": "2026-07-10T15:09:20.000000Z",
      "url": "http://127.0.0.1:8000/storage/products/gCCwI8vdo9b333MwFFyYZ4ISpYXbq9CGpFLrEdEO.jpg"
    }
  }
}
```

Evidencia CP-01 — petición y respuesta reales de la API.

ID	CP-02
Nombre	Listar productos registrados
Prioridad	Alta
Tipo de prueba	API
Endpoint	<code>GET /api/products</code>
Objetivo	Comprobar que index() devuelve la lista completa de productos con su imagen asociada.
Precondiciones	Existen productos en la base de datos.
Datos de entrada	Sin cuerpo. Solo la petición GET.
Pasos	1) Método GET y URL /api/products. 2) Ejecutar. 3) Revisar el arreglo JSON devuelto.
Resultado esperado	HTTP 200 con un arreglo de productos; cada elemento incluye su objeto image con la URL pública.
Resultado real	HTTP 200. Se devolvió el arreglo con todos los productos y su imagen (carga eager with('image')).
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-02

Listar productos registrados

GET http://127.0.0.1:8000/api/products

Status: **200 OK**

Herramienta: cURL

Content-Type: —

SOLICITUD (REQUEST)

Header: **Accept: application/json**

— sin cuerpo (no body) —

RESPUESTA (RESPONSE BODY - JSON)

```
[
  {
    "id": 2,
    "name": "Iphone 16",
    "description": "Smartphone Apple con chip A18 Pro",
    "price": "31999.00",
    "stock": 12,
    "created_at": "2026-07-10T02:04:23.000000Z",
    "updated_at": "2026-07-10T02:04:23.000000Z",
    "image": {
      "id": 2,
      "path": "products/h2IIhi3M2VWuqVo08AilPxFC8qjTwY3lePylqbqG.jpg",
      "imageable_type": "App\\Models\\Product",
      "imageable_id": 2,
      "created_at": "2026-07-10T02:04:23.000000Z",
      "updated_at": "2026-07-10T02:04:23.000000Z",
      "url": "http://127.0.0.1:8000/storage/products/h2IIhi3M2VWuqVo08AilPxFC8qjTwY3lePylqbqG.jpg"
    }
  },
  {
    "id": 3,
    "name": "Alexa",
    "description": "Bocina inteligente con Alexa, sonido mejorado y control por voz",
    "price": "1000.00",
    "stock": 35,
    "created_at": "2026-07-10T02:05:41.000000Z",
    "updated_at": "2026-07-10T02:05:41.000000Z",
    "image": {
      "id": 3,
      "path": "products/tlWLF8pEMaKk9glTYEjkyTOPkaj6qhzxzEtTRZjx.jpg",
      "imageable_type": "App\\Models\\Product",
      "imageable_id": 3,
      "created_at": "2026-07-10T02:05:41.000000Z",
      "updated_at": "2026-07-10T02:05:41.000000Z",
      "url": "http://127.0.0.1:8000/storage/products/tlWLF8pEMaKk9glTYEjkyTOPkaj6qhzxzEtTRZjx.jpg"
    }
  },
  // ... 8 productos mas - total de productos: 10
]
```

Evidencia CP-02 — petición y respuesta reales de la API.

ID	CP-03
Nombre	Consultar detalle de producto con imagen
Prioridad	Alta
Tipo de prueba	API
Endpoint	<code>GET /api/products/{id}</code>
Objetivo	Verificar que show() devuelve un producto específico junto con su imagen.
Precondiciones	El producto solicitado existe.
Datos de entrada	Parámetro de ruta id=11. Sin cuerpo.
Pasos	1) Método GET y URL /api/products/11. 2) Ejecutar. 3) Revisar el objeto JSON.
Resultado esperado	HTTP 200 con el producto y su objeto image completo (path + url accesible).
Resultado real	HTTP 200. Se devolvió el producto id=11 con su imagen y la URL pública generada por el accesor getUrlAttribute().
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-03

Consultar detalle de producto con imagen

GET http://127.0.0.1:8000/api/products/11

Status: **200 OK**

Herramienta: cURL

Content-Type: —

SOLICITUD (REQUEST)

Header: **Accept: application/json**

— sin cuerpo (no body) —

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "id": 11,
  "name": "Laptop Gamer Legion 5",
  "description": "Laptop para videojuegos con RTX 4060 y 16GB RAM",
  "price": "24999.90",
  "stock": 8,
  "created_at": "2026-07-10T15:09:20.000000Z",
  "updated_at": "2026-07-10T15:09:20.000000Z",
  "image": {
    "id": 11,
    "path": "products/gCCwI8vdo9b333MwFFyYZ4ISpYXbq9CGpFLrEdEO.jpg",
    "imageable_type": "App\\Models\\Product",
    "imageable_id": 11,
    "created_at": "2026-07-10T15:09:20.000000Z",
    "updated_at": "2026-07-10T15:09:20.000000Z",
    "url": "http://127.0.0.1:8000/storage/products/gCCwI8vdo9b333MwFFyYZ4ISpYXbq9CGpFLrEdEO.jpg"
  }
}
```

API REST de Productos - Laravel 12

Evidencia CP-03

Evidencia CP-03 — petición y respuesta reales de la API.

ID	CP-04
Nombre	Actualizar datos del producto
Prioridad	Alta
Tipo de prueba	API / Integración
Endpoint	<code>POST /api/products/{id}</code>
Objetivo	Comprobar que update() modifica los datos del producto sin perder la relación con su imagen.
Precondiciones	El producto id=11 existe.
Datos de entrada	form-data: name='Laptop Gamer Legion 5 Pro', price=27499.00, stock=5.
Pasos	1) Método POST y URL /api/products/11. 2) Enviar los campos a modificar. 3) Ejecutar y revisar la respuesta.
Resultado esperado	HTTP 200 con message='Producto actualizado correctamente.' y los datos actualizados; la imagen se conserva.
Resultado real	HTTP 200. Se actualizaron name, price y stock; la imagen previa se mantuvo intacta. (Ver también CP-09: cambio de imagen).
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-04

Actualizar datos del producto

POST http://127.0.0.1:8000/api/products/11

Status: **200 OK**

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: **Accept: application/json**

name	Laptop Gamer Legion 5 Pro	Text
price	27499.00	Text
stock	5	Text

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "message": "Producto actualizado correctamente.",
  "data": {
    "id": 11,
    "name": "Laptop Gamer Legion 5 Pro",
    "description": "Laptop para videojuegos con RTX 4060 y 16GB RAM",
    "price": "27499.00",
    "stock": 5,
    "created_at": "2026-07-10T15:09:20.000000Z",
    "updated_at": "2026-07-10T15:09:22.000000Z",
    "image": {
      "id": 11,
      "path": "products/gCCwI8vdo9b333MwFFyYZ4ISpYXbq9CGpFLrEdEO.jpg",
      "imageable_type": "App\\Models\\Product",
      "imageable_id": 11,
      "created_at": "2026-07-10T15:09:20.000000Z",
      "updated_at": "2026-07-10T15:09:20.000000Z",
      "url": "http://127.0.0.1:8000/storage/products/gCCwI8vdo9b333MwFFyYZ4ISpYXbq9CGpFLrEdEO.jpg"
    }
  }
}
```

API REST de Productos - Laravel 12

Evidencia CP-04

Evidencia CP-04 — petición y respuesta reales de la API.

CP-09

Actualizar imagen del producto

POST http://127.0.0.1:8000/api/products/11

Status: **200 OK**

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: **Accept: application/json**

image

valid_product.jpg

File

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "message": "Producto actualizado correctamente.",
  "data": {
    "id": 11,
    "name": "Laptop Gamer Legion 5 Pro",
    "description": "Laptop para videojuegos con RTX 4060 y 16GB RAM",
    "price": "27499.00",
    "stock": 5,
    "created_at": "2026-07-10T15:09:20.000000Z",
    "updated_at": "2026-07-10T15:09:22.000000Z",
    "image": {
      "id": 11,
      "path": "products/X8A6prQRCdZwkn5HR4djYpEiDWYGHS83ACclrxVb.jpg",
      "imageable_type": "App\\Models\\Product",
      "imageable_id": 11,
      "created_at": "2026-07-10T15:09:20.000000Z",
      "updated_at": "2026-07-10T15:09:23.000000Z",
      "url": "http://127.0.0.1:8000/storage/products/X8A6prQRCdZwkn5HR4djYpEiDWYGHS83ACclrxVb.jpg"
    }
  }
}
```

API REST de Productos · Laravel 12

Evidencia CP-09

Evidencia complementaria CP-09 — actualización de la imagen del mismo producto (HTTP 200).

ID	CP-05
Nombre	Eliminar producto
Prioridad	Alta
Tipo de prueba	API / Sistema
Endpoint	<code>DELETE /api/products/{id}</code>
Objetivo	Verificar que destroy() elimina el producto y controla la eliminación de su imagen (registro y archivo físico).
Precondiciones	El producto id=11 existe y tiene imagen.
Datos de entrada	Parámetro de ruta id=11. Sin cuerpo.
Pasos	1) Método DELETE y URL /api/products/11. 2) Ejecutar. 3) Consultar de nuevo el producto para confirmar (CP-05b).
Resultado esperado	HTTP 200 con message='Producto eliminado correctamente.'; posterior consulta del mismo id devuelve 404.
Resultado real	HTTP 200 al eliminar. La posterior consulta GET /api/products/11 devolvió 404 (ver CP-05b), confirmando el borrado del producto y su imagen.
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-05

Eliminar producto

DELETE

http://127.0.0.1:8000/api/products/11

Status: 200 OK

Herramienta: cURL

Content-Type: —

SOLICITUD (REQUEST)
Header: Accept: application/json
— sin cuerpo (no body) —

RESPUESTA (RESPONSE BODY · JSON)

```
{  
  "message": "Producto eliminado correctamente."  
}
```

API REST de Productos · Laravel 12

Evidencia CP-05

Evidencia CP-05 — petición y respuesta reales de la API.

CP-05b

Verificar que el producto ya no existe

GET

http://127.0.0.1:8000/api/products/11

Status: 404 Not Found

Herramienta: cURL

Content-Type: —

SOLICITUD (REQUEST)
Header: Accept: application/json
— sin cuerpo (no body) —

RESPUESTA (RESPONSE BODY · JSON)

```
{  
  "message": "No query results for model [App\\Models\\Product] 11",  
  "exception": "Symfony\\Component\\HttpKernel\\Exception\\NotFoundHttpException"  
}  
  
// (Laravel incluye además 'file', 'line' y 'trace' porque APP_DEBUG=true)
```

API REST de Productos · Laravel 12

Evidencia CP-05b

Evidencia complementaria CP-05b — la consulta posterior devuelve HTTP 404, confirmando el borrado.

ID	CP-06
Nombre	Crear producto sin campos obligatorios
Prioridad	Media
Tipo de prueba	Validación
Endpoint	<code>POST /api/products</code>
Objetivo	Verificar que la validación rechaza la creación cuando faltan campos requeridos.
Precondiciones	Servidor activo.
Datos de entrada	form-data: solo name='Producto incompleto' (faltan description, price, stock e image).
Pasos	1) Método POST y URL /api/products con header Accept: application/json. 2) Enviar solo name. 3) Ejecutar.
Resultado esperado	HTTP 422 con objeto 'errors' que lista los campos faltantes (description, price, stock, image).
Resultado real	HTTP 422. Se devolvieron los 4 errores de validación esperados sin crear ningún registro.
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-06

Crear producto sin campos obligatorios

POST http://127.0.0.1:8000/api/products

● Status: **422 Unprocessable Content**

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: **Accept:** application/json

name

Producto incompleto

Text

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "message": "The description field is required. (and 3 more errors)",
  "errors": {
    "description": [
      "The description field is required."
    ],
    "price": [
      "The price field is required."
    ],
    "stock": [
      "The stock field is required."
    ],
    "image": [
      "The image field is required."
    ]
  }
}
```

API REST de Productos - Laravel 12

Evidencia CP-06

Evidencia CP-06 — petición y respuesta reales de la API.

ID	CP-07
Nombre	Subir archivo que no es imagen
Prioridad	Media
Tipo de prueba	Validación
Endpoint	POST /api/products
Objetivo	Verificar que la regla image mimes rechaza un archivo que no es imagen.
Precondiciones	Servidor activo. Archivo no_es_imagen.txt disponible.
Datos de entrada	form-data: name, description, price, stock válidos + image=no_es_imagen.txt (text/plain).
Pasos	1) Método POST y URL /api/products con Accept: application/json. 2) Adjuntar el .txt en el campo image. 3) Ejecutar.
Resultado esperado	HTTP 422 con errores indicando que 'image' debe ser una imagen de tipo jpg, jpeg o png.
Resultado real	HTTP 422. La API rechazó el archivo con los mensajes 'must be an image' y 'must be a file of type: jpg, jpeg, png'.
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

CP-07

Subir archivo que no es imagen

POST http://127.0.0.1:8000/api/products

● Status: **422 Unprocessable Content**

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: Accept: application/json

name	Producto con archivo malo	Text
description	Prueba de validacion	Text
price	100	Text
stock	3	Text
image	 no_es_imagen.txt	File

RESPUESTA (RESPONSE BODY - JSON)

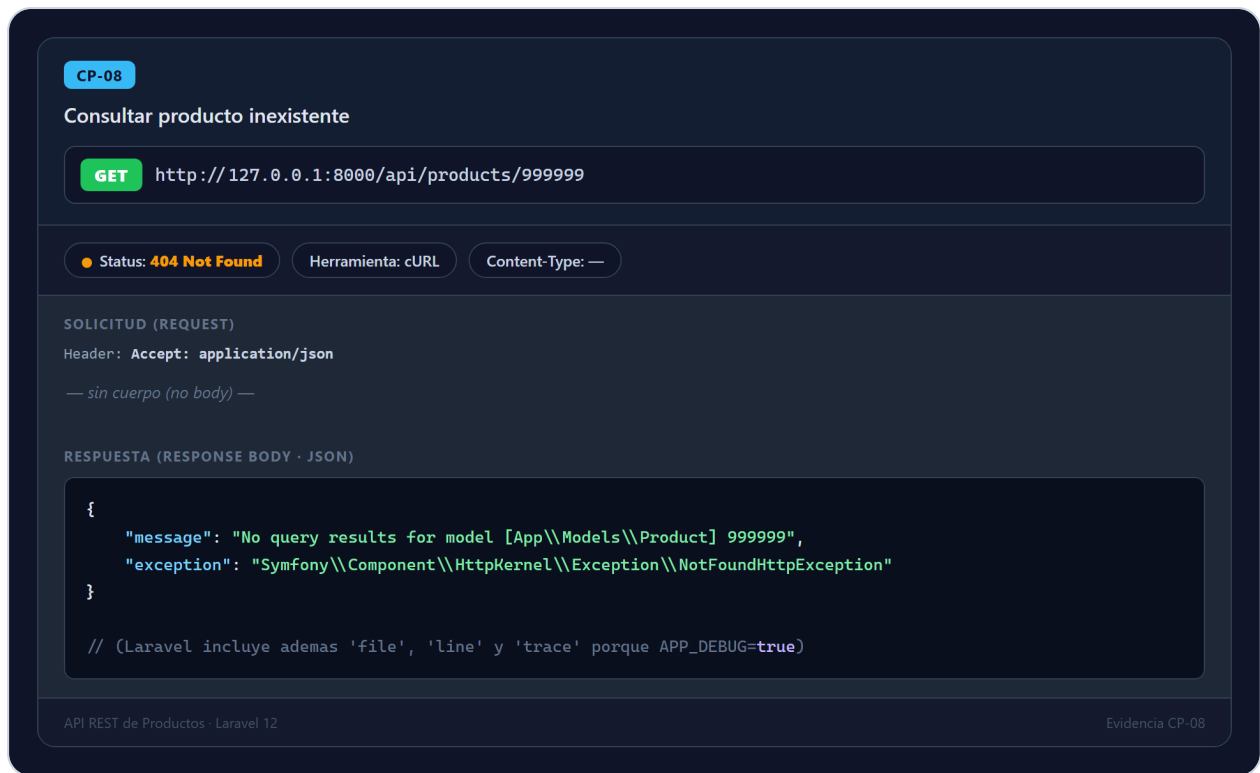
```
{
  "message": "The image field must be an image. (and 1 more error)",
  "errors": {
    "image": [
      "The image field must be an image.",
      "The image field must be a file of type: jpg, jpeg, png."
    ]
  }
}
```

API REST de Productos - Laravel 12

Evidencia CP-07

Evidencia CP-07 — petición y respuesta reales de la API.

ID	CP-08
Nombre	Consultar producto inexistente
Prioridad	Media
Tipo de prueba	API / Validación
Endpoint	<code>GET /api/products/{id}</code>
Objetivo	Verificar que consultar un id que no existe devuelve 404 y no un error 500.
Precondiciones	No existe un producto con id=999999.
Datos de entrada	Parámetro de ruta id=999999. Sin cuerpo.
Pasos	1) Método GET y URL /api/products/999999 con Accept: application/json. 2) Ejecutar.
Resultado esperado	HTTP 404 (NotFoundException) generado por findOrFail(), con mensaje descriptivo.
Resultado real	HTTP 404. findOrFail() lanzó la excepción y Laravel respondió 404 correctamente (no 500).
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.



Evidencia CP-08 — petición y respuesta reales de la API.

Casos adicionales

CP-09 (actualización de imagen) se muestra junto a CP-04. A continuación, CP-10:

ID	CP-10
Nombre	Crear producto con precio y stock inválidos
Tipo	Validación
Endpoint	<code>POST /api/products</code>
Objetivo	Verificar que $\text{price} \geq 0$ y $\text{stock} \geq 0$ se validan correctamente.
Datos de entrada	form-data: price=-50, stock=-3 (más name, description, image válidos).
Resultado esperado	HTTP 422 con errores en price y stock indicando el mínimo 0.
Resultado real	HTTP 422. Se rechazaron ambos campos con 'must be at least 0'.
Estado	✓ Aprobado

CP-10

Crear producto con precio y stock invalidos

POST http://127.0.0.1:8000/api/products

● Status: **422 Unprocessable Content**

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: Accept: application/json

name	Producto precio invalido	Text
description	Precio negativo	Text
price	-50	Text
stock	-3	Text
image	 valid_product.jpg	File

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "message": "The price field must be at least 0. (and 1 more error)",
  "errors": {
    "price": [
      "The price field must be at least 0."
    ],
    "stock": [
      "The stock field must be at least 0."
    ]
  }
}
```

Evidencia CP-10.

10. Matriz de seguimiento y control (Plantilla 12.3)

ID	Caso	Prioridad	Tipo	Método	Estado	Evidencia
CP-01	Crear producto con imagen	Alta	API / Funcional	cURL / Manual	✓ Aprobado	HTTP 201 + JSON
CP-02	Listar productos	Alta	API	cURL / Manual	✓ Aprobado	HTTP 200 + arreglo
CP-03	Consultar producto por id	Alta	API	cURL / Manual	✓ Aprobado	HTTP 200 + imagen
CP-04	Actualizar producto	Alta	API / Integración	cURL / Manual	✓ Aprobado	HTTP 200
CP-05	Eliminar producto	Alta	API / Sistema	cURL / Manual	✓ Aprobado	HTTP 200 + 404 posterior
CP-06	Validar campos obligatorios	Media	Validación	cURL / Manual	✓ Aprobado	HTTP 422 (JSON de errores)
CP-07	Rechazar archivo no imagen	Media	Validación	cURL / Manual	✓ Aprobado	HTTP 422 (archivo inválido)
CP-08	Consultar producto inexistente	Media	API / Validación	cURL / Manual	✓ Aprobado	HTTP 404
CP-09	Actualizar imagen del producto	Media	API / Integración	cURL / Manual	✓ Aprobado	HTTP 200 (nuevo path)
CP-10	Precio / stock inválidos	Media	Validación	cURL / Manual	✓ Aprobado	HTTP 422

Resumen: 10 casos ejecutados · **10 aprobados** · 0 rechazados · 0 bloqueados. Cobertura del 100% de los endpoints y de las validaciones mínimas exigidas.

11. Reporte de defectos (Plantilla 12.4)

Aunque todos los casos de prueba se aprobaron a nivel funcional, la revisión del código detectó tres observaciones de calidad que se documentan como defectos de severidad baja/media:

ID del defecto	D-01
Título	Import con mayúsculas/minúsculas incorrectas en el namespace (<code>use App\models\Image;</code>).
Descripción	En <code>ProductController.php</code> (línea 7) se importa <code>App\models\Image</code> con <code>models</code> en minúscula, cuando el namespace real es <code>App\Models</code> . Además el import no se utiliza (la relación se usa vía <code>\$product->image()</code>).
Pasos para reproducir	Desplegar el proyecto en un sistema de archivos sensible a mayúsculas (Linux) y referenciar la clase <code>Image</code> importada.
Resultado esperado	El import debería ser <code>App\Models\Image</code> , o eliminarse por no usarse.
Resultado real	En Windows funciona (sistema no sensible a mayúsculas); en Linux fallaría el autoload si la clase se usara.
Severidad	Baja — no afecta la ejecución actual, pero es un riesgo de portabilidad y código muerto.
Estado	Nuevo — recomendación: corregir a <code>App\Models\Image</code> o eliminar el import.

ID del defecto	D-02
Título	La actualización no responde a <code>PUT/PATCH</code> , solo a <code>POST</code> .
Descripción	El examen sugiere <code>PUT/PATCH /api/products/{id}</code> , pero solo se registró <code>POST /api/products/{id}</code> . Una petición PUT/PATCH devolvería <code>405 Method Not Allowed</code> .
Pasos para reproducir	Enviar <code>PUT /api/products/11</code> con datos de actualización.
Resultado esperado	Aceptar PUT/PATCH (o documentar el uso de POST con <i>method spoofing</i> <code>_method=PUT</code>).
Resultado real	Solo POST es aceptado; PUT/PATCH no está enrutado.
Severidad	Media — desviación respecto a la ruta sugerida, justificada por la subida de archivos en <code>multipart/form-data</code> .
Estado	Nuevo — recomendación: añadir <code>Route::match(['put','patch','post'], ...)</code> o soportar <code>_method=PUT</code> en el formulario.

ID del defecto	D-03
Título	Tipos de datos inconsistentes en la respuesta de <code>store()</code> .
Descripción	Al crear (CP-01), <code>price</code> y <code>stock</code> se devuelven como cadenas (<code>"24999.90"</code> , <code>"8"</code>), mientras que al consultar (CP-03) <code>stock</code> regresa como entero (<code>8</code>). Se debe a que el modelo <code>Product</code> no define <code>\$casts</code> .
Pasos para reproducir	Comparar el JSON de <code>POST /api/products</code> con el de <code>GET /api/products/{id}</code> .
Resultado esperado	Tipos consistentes (por ejemplo <code>stock</code> siempre entero y <code>price</code> siempre decimal).
Resultado real	La respuesta de creación conserva los tipos de <code>form-data</code> (cadenas).
Severidad	Baja — cosmético; se corrige agregando <code>protected \$casts = ['price'⇒'decimal:2', 'stock'⇒'integer']</code> .
Estado	Nuevo — recomendación: definir <code>\$casts</code> en el modelo <code>Product</code> .

12. Tipos de prueba propuestos

Se proponen y justifican cuatro tipos de prueba para la API de productos:

Prueba unitaria

Qué comprueba: que la relación `Product::image()` devuelve una instancia de `morphOne` hacia `Image` , y que las reglas de `StoreProductRequest` rechazan `price` negativo.

Por qué importa: valida en aislamiento la pieza más pequeña (modelo y validación) sin depender de HTTP ni de la base de datos; si esto falla, todo lo demás falla. Se implementaría con PHPUnit aserciones sobre `$product→image()` y sobre el `Validator` .

Prueba de integración

Qué comprueba: que al ejecutar `store()` se guarda un registro en `products` y uno en `images` con `imageable_id/imageable_type` correctos, usando `RefreshDatabase` y `Storage::fake()`.

Por qué importa: verifica que varias partes (controlador + Eloquent + almacenamiento + BD) trabajan juntas y que la imagen queda realmente asociada al producto. Corresponde a los casos CP-01 y CP-09.

Prueba de sistema

Qué comprueba: el flujo completo de la API en el entorno configurado: crear (CP-01) → listar (CP-02) → consultar (CP-03) → actualizar (CP-04) → eliminar (CP-05), ejecutado con cURL contra `php artisan serve`.

Por qué importa: confirma que la API funciona de punta a punta como un sistema real, con su base de datos PostgreSQL y su almacenamiento, tal como la usaría un cliente. Es exactamente lo evidenciado en la sección 9.

Prueba de aceptación

Qué comprueba: que el módulo cumple el criterio del negocio: “el usuario puede administrar productos con una imagen principal”. Se valida que cada producto listado muestra su imagen (sección 7) y que las operaciones CRUD responden con los códigos y mensajes esperados.

Por qué importa: traduce los requisitos del examen a criterios verificables; si se cumplen, el entregable se considera aceptado por el usuario final.

13. Conclusión técnica

Qué funcionó. Los cinco endpoints operan correctamente y devuelven JSON con los códigos HTTP adecuados: 201 al crear, 200 al listar/consultar/actualizar/eliminar, 422 en validaciones y 404 en recursos inexistentes. La relación polimórfica `morphOne/morphTo` quedó verificada tanto en los datos (campos `imageable_id` / `imageable_type`) como visualmente (galería de la sección 7). El manejo de la imagen —subida, asociación, reemplazo en update y borrado del archivo físico en destroy— se comportó según lo esperado, y las transacciones evitan estados inconsistentes.

Qué errores se encontraron. A nivel funcional, ninguno bloqueante: los 10 casos de prueba se aprobaron. La revisión de código detectó tres observaciones de calidad (D-01 casing del import, D-02 ausencia de ruta PUT/PATCH, D-03 tipos inconsistentes por falta de `$casts`), todas de severidad baja/media y con corrección propuesta.

Qué se aprendió sobre pruebas de API. Probar una API va más allá de “que responda”: implica verificar el código HTTP, la estructura del JSON, el efecto real en la base de datos y el manejo de archivos, además de los caminos de error (campos faltantes, archivo inválido, id inexistente). Distinguir los niveles de prueba (unitaria, integración, sistema y aceptación) ayuda a localizar dónde puede fallar el sistema.

Por qué el plan de pruebas mejora la calidad. Diseñar casos con datos correctos e incorrectos, registrar la evidencia y llevar una matriz de seguimiento convierte la verificación en un proceso repetible y auditable. Así se detectan defectos temprano, se documenta el estado real del software y se sustenta —con pruebas— que el módulo cumple lo solicitado.